Pemodelan dengan Convolutional Neural Network (CNN)

Disusun dalam Rangka Memenuhi Tugas Ujian Akhir Semester Mata Kuliah Analisis Data Tidak Terstruktur

Semester Genap Tahun Akademik 2025/2026

Dosen Pengampu:

Abdul Kudus, S.SI., M.SI., PH.D.

Disusun Oleh:

Keysha Jauzi Althaff 10060122019

PROGRAM STUDI STATISTIKA

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM (MIPA)

UNIVERSITAS ISLAM BANDUNG

2025

SOAL

Diberikan data berupa kumpulan gambar karakter kartun Homer dan Bart Simpson di link http://bit.ly/3HIwP3d

letaknya dalam folder "Datasets" dengan nama file "homer_bart_2.zip"

Lakukan pemodelan dengan Convolutional Neural Network (CNN) untuk memprediksi apakah gambar yang diinputkan adalah “Bart” atau “Homer”?

Langkah-langkah yang harus dilaporkan:

1. Baca gambar **(CPMK01, SKOR 15)**
2. Split data menjadi Data Training dan Data Testing **(CPMK02, SKOR 10)**
3. Buat CNN dengan Data Training dengan tahapan: **(CPMK03, SKOR 50)**
    a. Lakuan feature detector yang menghasilkan feature maps
    b. Lakukan pooling
    c. Lakukan flattening
    d. Lakukan dense neural network
4. Evaluasi model yang sudah dibuat dengan Data Testing dan laporkan matrix confusion yang diperoleh **(CPMK04, SKOR 25)**

Keterangan:
Jawaban harus disertai dengan script program komputer.

PENYELESAIAN

- **Import Library**

Syntax dan Output

Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Bagikan Gemini

Perintah + Kode + Teks Jalankan semua

Convolutional neural network for image classification

Importing the libraries

```
import matplotlib.pyplot as plt
import seaborn as sns
import zipfile
import numpy as np
import cv2
from google.colab.patches import cv2_imshow
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator
tf.__version__
```

```
'2.18.0'
```

Loading the images

Variabel Terminal 19.19 Python 3

Interpretasi

Syntax ini digunakan untuk mengimport library penting yang akan digunakan untuk melakukan pemodelan dengan Convolutional Neural Network (CNN), dan membantu mengetahui versi TensorFlow yang sedang digunakan agar tahu kompatibilitas fitur.

1. Baca gambar

Syntax dan Output

Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Bagikan Gemini

Perintah + Kode + Teks Jalankan semua

Loading the images

```
[4] from google.colab import drive
    drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
[5] path = '/content/drive/MyDrive/ADTT/Computer Vision Masterclass-20250420T104721Z-001/Computer Vision Masterclass/Datasets/homer_bart_2.zip'
    zip_object = zipfile.ZipFile(file=path, mode='r')
    zip_object.extractall('./')
    zip_object.close()
```

```
tf.keras.preprocessing.image.load_img('/content/homer_bart_2/training_set/bart/bart100.bmp')
```



Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Bagikan Gemini

Perintah + Kode + Teks Jalankan semua

RAM Disk

```
[7] tf.keras.preprocessing.image.load_img('/content/homer_bart_2/training_set/homer/homer100.bmp')
```



Variabel Terminal

19.44 Python 3

Interpretasi
Menghubungkan Google Collab dengan Google Drive, setelah itu mengambil dataset gambar dalam format .zip dari Google Drive dan mengekstraknya ke Colab, lalu mengecek apakah file terbaca dengan benar.

2. Split data menjadi Data Training dan Data Testing

Syntax dan Output

Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Bagikan Gemini

Perintah + Kode + Teks Jalankan semua

RAM Disk

Train and test set

```
[8] training_generator = ImageDataGenerator(rescale=1./255,
                                            rotation_range=7,
                                            horizontal_flip=True,
                                            zoom_range=0.2)

train_dataset = training_generator.flow_from_directory('/content/homer_bart_2/training_set',
                                                       target_size = (64, 64),
                                                       batch_size = 8,
                                                       class_mode = 'categorical',
                                                       shuffle = True)
```

```
Found 215 images belonging to 2 classes.
```

Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Bagikan Gemini

Perintah + Kode + Teks Jalankan semua

RAM Disk

```
train_dataset.classes
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
      dtype=int32)
```

```
[11] train_dataset.class_indices
```

```
{'bart': 0, 'homer': 1}
```

```
[13] test_generator = ImageDataGenerator(rescale=1./255)

[14] test_dataset = test_generator.flow_from_directory('/content/homer_bart_2/test_set',
                                                       target_size = (64, 64),
                                                       batch_size = 1,
                                                       class_mode = 'categorical',
                                                       shuffle = False)
```

```
Found 54 images belonging to 2 classes.
```

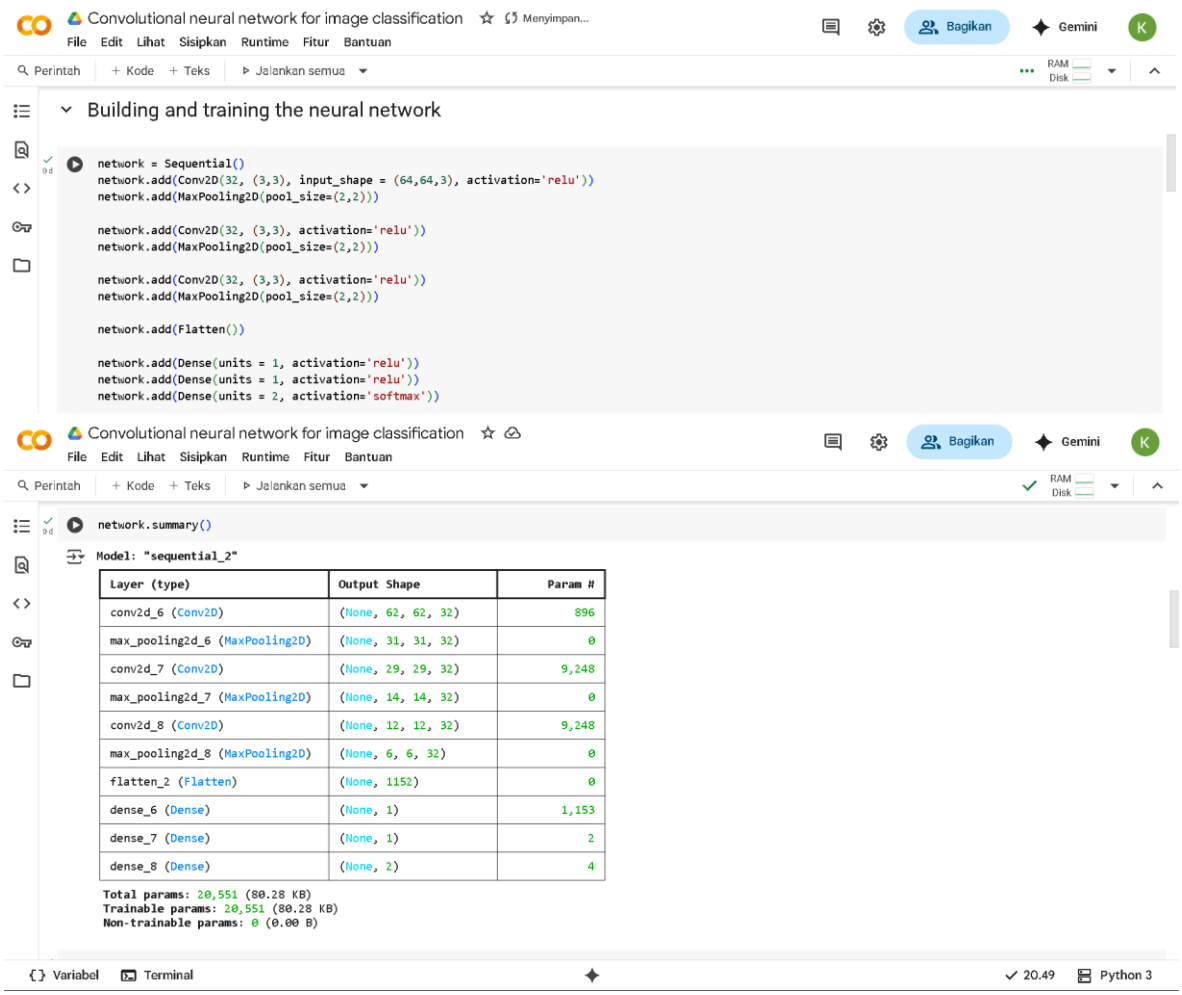
Variabel Terminal

20.07 Python 3

Interpretasi
Dataset training diproses dengan augmentasi berupa normalisasi piksel, rotasi acak 7°, pembalikan horizontal, dan zoom 20% untuk meningkatkan variasi gambar. Data kemudian diambil dari folder /training_set, diubah ukurannya menjadi 64x64 piksel, dibaca dalam batch berisi 8 gambar, dan dikategorikan secara otomatis berdasarkan nama folder, dengan urutan label acak. Label numerik dari setiap gambar dapat dilihat melalui train_dataset.classes, sedangkan train_dataset.class_indices menunjukkan mapping nama kelas ke label angka. Sementara itu, dataset testing diproses dengan normalisasi saja tanpa augmentasi, diambil dari folder /test_set, diubah ke ukuran yang sama (64x64), dibaca satu per satu (batch_size=1), dan tidak diacak (shuffle=False) agar urutan hasil prediksi sesuai dengan urutan file aslinya.

3. Buat CNN dengan Data Training dengan tahapan: **(CPMK03, SKOR 50)**
    e. Lakuan feature detector yang menghasilkan feature maps
    f. Lakukan pooling
    g. Lakukan flattening
    h. Lakukan dense neural network

Syntax dan Output



Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Perintah + Kode + Teks ▶ Jalankan semua

Bagikan Gemini

```
[24] (1152+2)/2
```

```
577.0
```

```
network = Sequential()
network.add(Conv2D(32, (3,3), input_shape = (64,64,3), activation='relu'))
network.add(MaxPooling2D(pool_size=(2,2)))

network.add(Conv2D(32, (3,3), activation='relu'))
network.add(MaxPooling2D(pool_size=(2,2)))

network.add(Conv2D(32, (3,3), activation='relu'))
network.add(MaxPooling2D(pool_size=(2,2)))

network.add(Flatten())

network.add(Dense(units = 577, activation='relu'))
network.add(Dense(units = 577, activation='relu'))
network.add(Dense(units = 2, activation='softmax'))
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a la
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Perintah + Kode + Teks ▶ Jalankan semua

Bagikan Gemini

```
network.summary()
```

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d_9 (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_10 (Conv2D)	(None, 29, 29, 32)	9,248
max_pooling2d_10 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_11 (Conv2D)	(None, 12, 12, 32)	9,248
max_pooling2d_11 (MaxPooling2D)	(None, 6, 6, 32)	0
flatten_3 (Flatten)	(None, 1152)	0
dense_9 (Dense)	(None, 577)	665,281
dense_10 (Dense)	(None, 577)	333,506
dense_11 (Dense)	(None, 2)	1,156

```
Total params: 1,019,335 (3.89 MB)
Trainable params: 1,019,335 (3.89 MB)
Non-trainable params: 0 (0.00 B)
```

Variabel Terminal 20.49 Python 3

Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Perintah + Kode + Teks ▶ Jalankan semua

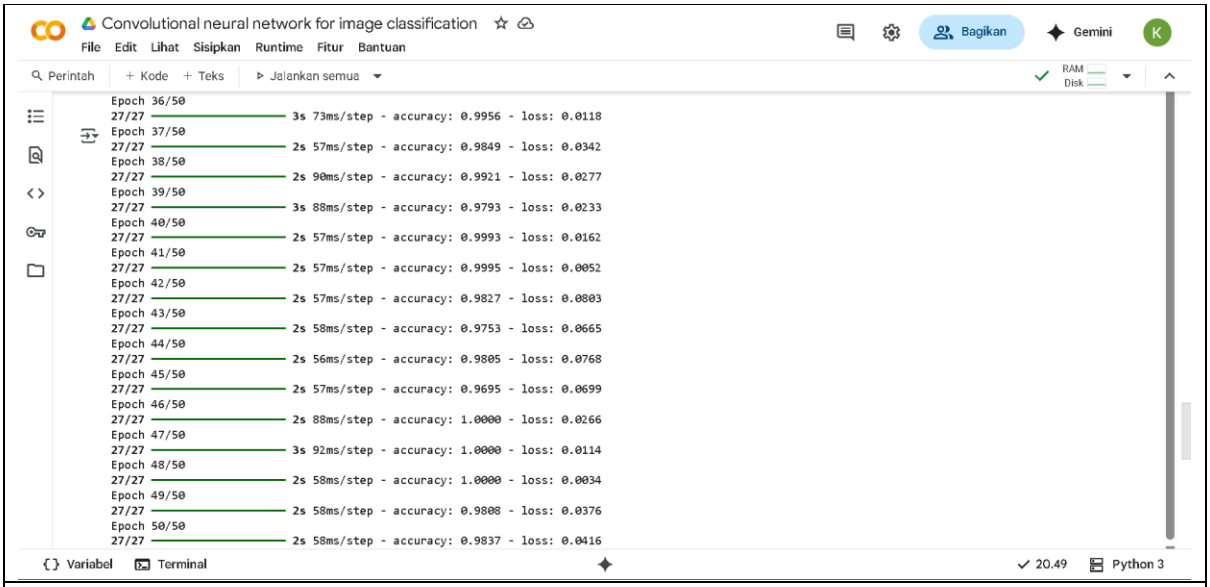
Bagikan Gemini

```
[29] network.compile(optimizer='Adam', loss='categorical_crossentropy', metrics = ['accuracy'])
```

```
history = network.fit(train_dataset, epochs=50)
```

```
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 56ms/step - accuracy: 0.9790 - loss: 0.0550
Epoch 23/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 79ms/step - accuracy: 0.9933 - loss: 0.0382
Epoch 24/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 3s 91ms/step - accuracy: 0.9743 - loss: 0.0663
Epoch 25/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 57ms/step - accuracy: 0.9629 - loss: 0.1003
Epoch 26/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 57ms/step - accuracy: 0.9892 - loss: 0.0272
Epoch 27/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 57ms/step - accuracy: 0.9862 - loss: 0.0484
Epoch 28/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 56ms/step - accuracy: 0.9946 - loss: 0.0214
Epoch 29/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 56ms/step - accuracy: 0.9571 - loss: 0.1044
Epoch 30/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 56ms/step - accuracy: 0.9454 - loss: 0.1613
Epoch 31/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 85ms/step - accuracy: 0.9728 - loss: 0.0821
Epoch 32/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 3s 107ms/step - accuracy: 0.9156 - loss: 0.2015
Epoch 33/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 59ms/step - accuracy: 0.9814 - loss: 0.0421
Epoch 34/50
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 57ms/step - accuracy: 0.9883 - loss: 0.0353
```

Variabel Terminal 20.49 Python 3



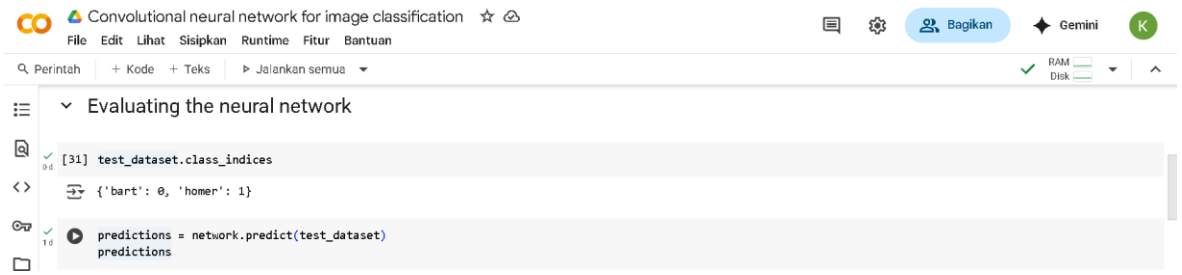
Interpretasi

Model CNN dibangun secara berurutan menggunakan Sequential():

- Tahap Feature Detector:
  network.add(Conv2D(...))
  Tiga layer Conv2D digunakan untuk mendeteksi fitur visual dari gambar dan menghasilkan feature maps.
- Tahap Pooling:
  network.add(MaxPooling2D(...))
  Setiap layer konvolusi diikuti oleh MaxPooling2D untuk menyederhanakan feature maps dan mengurangi ukuran data.
- Tahap Flattening:
  network.add(Flatten())
  Layer ini mengubah hasil pooling dari bentuk 2D menjadi 1D agar bisa diproses oleh dense layer.
- Tahap Dense Neural Network:
  network.add(Dense(...))
  Tiga layer Dense digunakan, dua dengan aktivasi ReLU sebagai hidden layer, dan satu dengan softmax untuk klasifikasi dua kelas.

4. Evaluasi model yang sudah dibuat dengan Data Testing dan laporkan matrix confusion yang diperoleh

Syntax dan Output



Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Bagikan Gemini

Perintah + Kode + Teks Jalankan semua

```
54/54 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step
array([[1.00000000e+00, 2.40160257e-16],
       [1.00000000e+00, 3.38785640e-08],
       [3.23133869e-03, 9.96768713e-01],
       [1.00000000e+00, 1.06790930e-18],
       [1.00000000e+00, 1.47950393e-18],
       [1.00000000e+00, 3.44702808e-19],
       [9.99981284e-01, 1.86857687e-05],
       [9.94493008e-01, 5.50704077e-03],
       [9.84782517e-01, 1.52175063e-02],
       [8.45295429e-01, 1.54704586e-01],
       [9.44599688e-01, 5.54003455e-02],
       [1.00000000e+00, 5.73171718e-12],
       [9.99973178e-01, 2.68767690e-05],
       [9.99998808e-01, 1.17248555e-06],
       [1.00000000e+00, 9.34340786e-18],
       [9.99960661e-01, 3.93314440e-05],
       [9.58037913e-01, 4.19620238e-02],
       [9.99960184e-01, 3.98168013e-05],
       [9.97936368e-01, 2.06370768e-03],
       [1.00000000e+00, 4.01733270e-21],
       [5.17585754e-01, 4.82414156e-01],
       [1.00000000e+00, 9.21678813e-24],
       [1.00000000e+00, 1.04400349e-10],
       [9.82601464e-01, 1.73985809e-02],
       [9.99999762e-01, 2.84434265e-07],
       [1.00000000e+00, 1.78370918e-08],
       [9.99985218e-01, 1.47964993e-05],
       [1.00000000e+00, 1.35409270e-14],
       [2.99387084e-05, 9.99970078e-01],
       [4.70728082e-05, 9.99952912e-01],
       [4.50082531e-08, 1.00000000e+00],
       [1.94143050e-03, 9.98058498e-01],
       [9.99904513e-01, 9.54570496e-05],
       [3.18445236e-04, 9.99681592e-01],
       [9.99894977e-01, 1.04964311e-04],
       [2.54321230e-06, 9.99997497e-01],
       [6.06538903e-04, 9.99393463e-01],
       [2.50089352e-06, 9.99997497e-01],
       [8.14595958e-04, 9.99185383e-01],
       [1.27848745e-08, 1.00000000e+00],
       [8.25275777e-07, 9.99999166e-01],
       [6.27798863e-06, 9.99993682e-01],
       [1.39309272e-01, 8.60690653e-01],
       [1.24285193e-02, 9.87571478e-01],
       [9.93862271e-01, 6.13768771e-03],
       [1.23742648e-04, 9.99876261e-01],
       [5.89860022e-01, 4.10139978e-01],
       [8.46431591e-10, 1.00000000e+00],
       [1.88760197e-04, 9.99811232e-01],
       [2.98134178e-01, 7.01865852e-01],
       [1.02900344e-06, 9.99998927e-01],
       [1.04260156e-02, 9.89574015e-01],
       [6.17939862e-04, 9.99382019e-01],
       [8.56814779e-07, 9.99999166e-01]], dtype=float32)
```

[33]
```
predictions = np.argmax(predictions, axis = 1)
predictions
```

```
array([0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1,
       0, 1, 0, 1, 1, 1, 1, 1, 1, 1])
```

[34]
```
test_dataset.classes
```

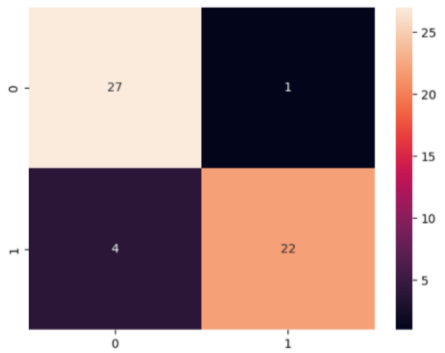
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=int32)
```

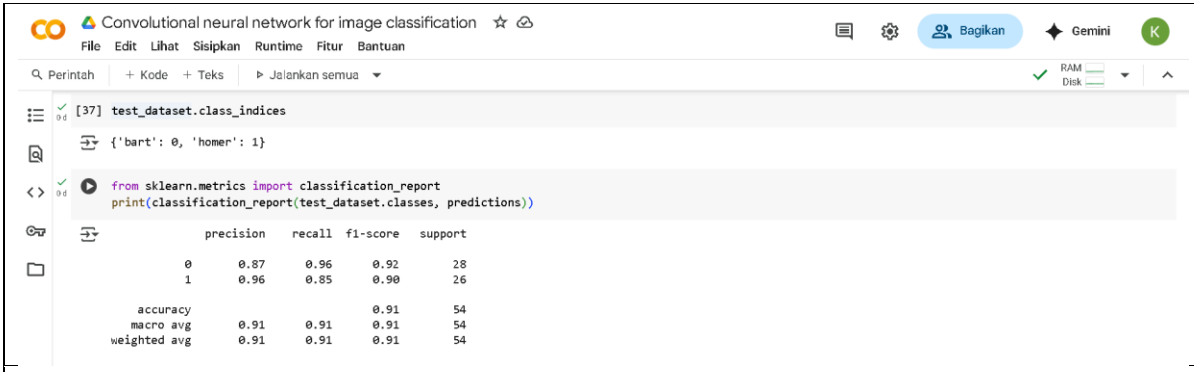
[35]
```
from sklearn.metrics import accuracy_score
accuracy_score(test_dataset.classes, predictions)
```

```
0.9074074074074074
```

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(test_dataset.classes, predictions)
sns.heatmap(cm, annot=True)
```

```
<Axes: >
```





Interpretasi

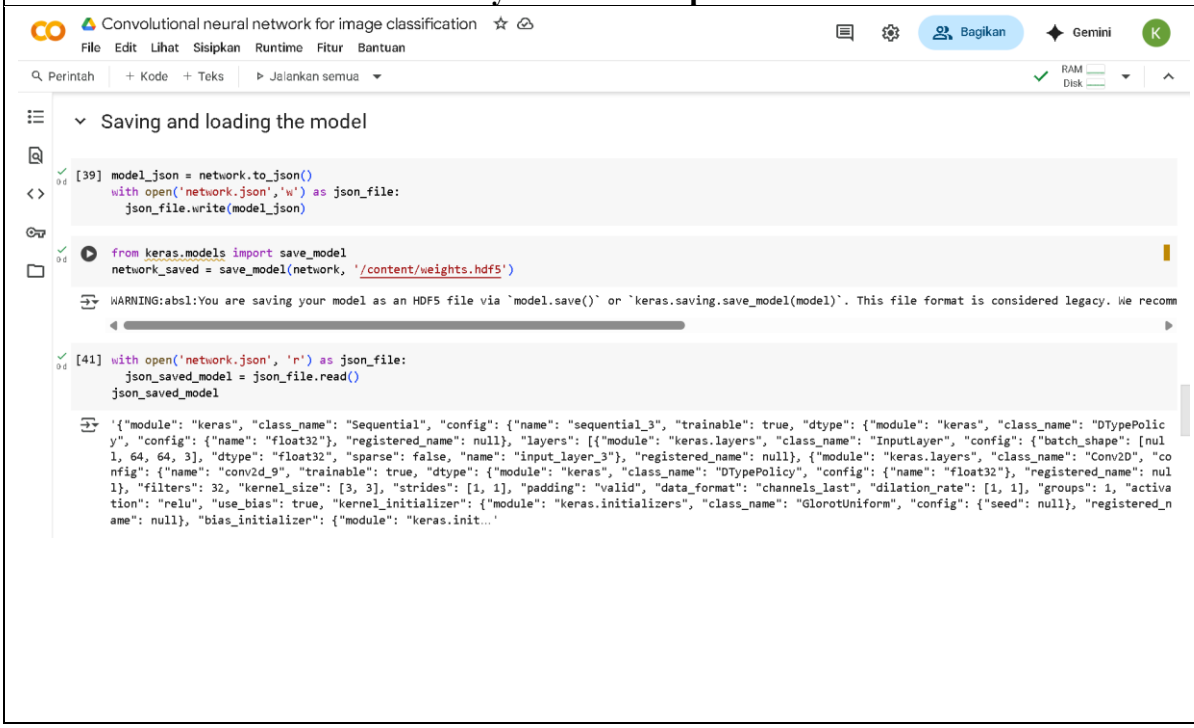
Model CNN telah dievaluasi menggunakan data testing yang terdiri dari 54 gambar (28 gambar Bart dan 26 gambar Homer). Evaluasi dilakukan dengan memprediksi data menggunakan model, kemudian membandingkan hasil prediksi dengan label aslinya.
Hasil evaluasi menunjukkan akurasi model sebesar 90.74%, yang menunjukkan bahwa model cukup baik dalam membedakan kedua kelas.
Confusion matrix yang diperoleh adalah sebagai berikut:

	Prediksi Bart (0)	Prediksi Homer (1)
Aktual Bart (0)	27 (benar)	1 (salah)
Aktual Homer (1)	4 (salah)	22 (benar)

- Model berhasil mengenali 27 dari 28 gambar Bart dengan benar.
- Model juga berhasil mengenali 22 dari 26 gambar Homer.
- 1 gambar Bart salah dikenali sebagai Homer.
- 4 gambar Homer salah dikenali sebagai Bart.

- **Saving and Loading the Model**

Syntax dan Output



```
[42] network_loaded = tf.keras.models.model_from_json(json_saved_model)
     network_loaded.load_weights('weights.hdf5')
     network_loaded.compile(loss = 'categorical_crossentropy', optimizer='Adam', metrics=['accuracy'])

     network_loaded.summary()

Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d_9 (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_10 (Conv2D)	(None, 29, 29, 32)	9,248
max_pooling2d_10 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_11 (Conv2D)	(None, 12, 12, 32)	9,248
max_pooling2d_11 (MaxPooling2D)	(None, 6, 6, 32)	0
flatten_3 (Flatten)	(None, 1152)	0
dense_9 (Dense)	(None, 577)	665,281
dense_10 (Dense)	(None, 577)	333,506
dense_11 (Dense)	(None, 2)	1,156

```
Total params: 1,019,335 (3.89 MB)
Trainable params: 1,019,335 (3.89 MB)
Non-trainable params: 0 (0.00 B)
```

Interpretasi

Section ini digunakan untuk menyimpan model CNN yang telah dilatih agar dapat digunakan kembali tanpa perlu melatih ulang. Model disimpan dan setelah itu, model dapat dimuat kembali untuk keperluan evaluasi dan prediksi. Hal ini sangat berguna untuk menghemat waktu dan memastikan reusabilitas model.

- **Classifying One Single Image**

Syntax dan Output

Convolutional neural network for image classification

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Bagikan Gemini

Perintah + Kode + Teks Jalankan semua

RAM Disk

Classifying one single image

```
image = cv2.imread('/content/homer_bart_2/test_set/homer/homer15.bmp')
```

```
[45] cv2_imshow(image)
```



```
[46] image = cv2.resize(image, (64, 64))
     cv2_imshow(image)
```



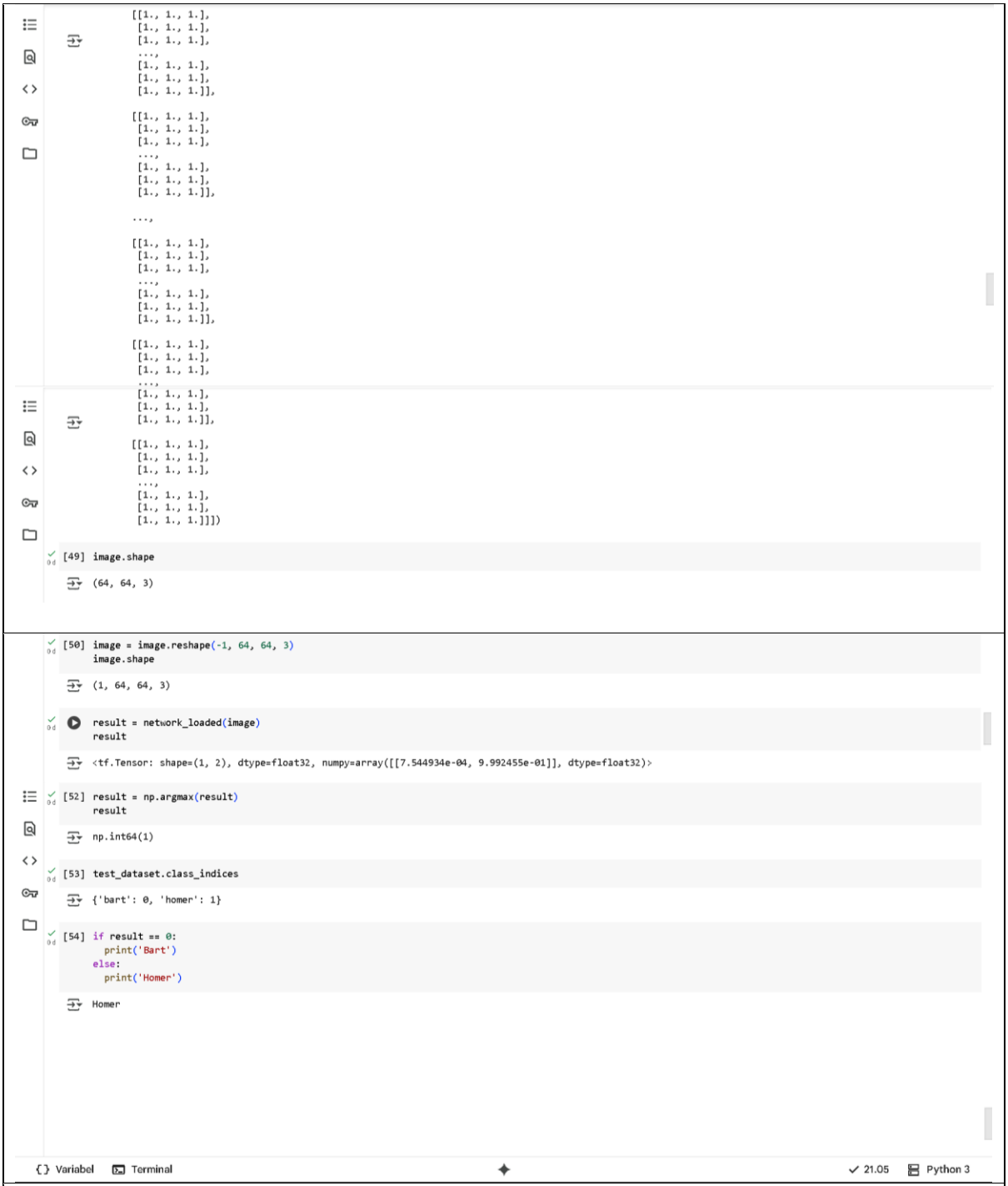
```
[47] image
```

ndarray (64, 64, 3) show data



```
image = image / 255
image
```

```
array([[[1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.],
        ...,
        [1., 1., 1.],
        [1., 1., 1.],
        [1., 1., 1.]],
```



Interpretasi

digunakan untuk mengklasifikasikan satu gambar secara manual. Gambar dibaca, diubah ukurannya ke 64x64, dinormalisasi, dan di-reshape agar sesuai dengan input model. Setelah diprediksi, hasilnya dikonversi ke kelas (bart atau homer) berdasarkan nilai probabilitas tertinggi. Output menunjukkan gambar tersebut dikenali sebagai Homer.

LAMPIRAN

Link Google Collab:

https://colab.research.google.com/drive/1UhDbpsDkSM_Q1bBqWee5P6Qj4LJo3NPQ?usp=sharing